介绍

这次练习中，我们将使用葡萄酒质量数据集。该数据集包含葡萄酒的各种化学性质，如酸度、糖分、PH值和酒精含量等，还包括两列分别表示葡萄酒的质量（3-9，越高越好）和酒的颜色（红或者白）。数据保存在文件中。 `Wine_Quality_Data.csv`

第一步：导入与处理数据

- 导入数据
- 绘制数据集的 `pairplot`
- 用列创建目标变量，为只取0/1二值的列，1表示“red” `color y y`
- 计算每列与的相关性，并显示成一个柱形图 `y`
- 挑选出与最相关（相关性的绝对值最大）的两列生成 `y X`
- 用MinMaxScaler缩放。注意：缩放后输出的是一个np.array，将它重新变成一个DataFrame `X`

在 [19]…

```
# read Wine_Quality_Data.csv into a DataFrame

import pandas as pd
data = pd.read_csv("Wine_Quality_Data.csv")
data.head()
```

出[19]:

	fixed_acidity	volatile_acidity	citric_acid	residual_sugar	氯化物	free_sulfur_dioxic
0	7.4	0.70	0.00	1.9	0.076	11
1	7.8	0.88	0.00	2.6	0.098	25
2	7.8	0.76	0.04	2.3	0.092	15
3	11.2	0.28	0.56	1.9	0.075	17
4	7.4	0.70	0.00	1.9	0.076	11

在 [2] …

```
# draw pairplot

import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

sns.pairplot(data, hue='color')
```

输出[2]:

```
<seaborn.axisgrid.PairGrid at 0x854e7c0>
```

在 [20]…

```
# create y

y = data.color.map(lambda x: 1 if x=='red' else 0)
```

在 [21]…

```
# compute correlation of each column with "color"

fields = list(data.columns[:-1])
correlations = data[fields].corrwith(y)
correlations.sort_values(inplace=True)
correlations
```

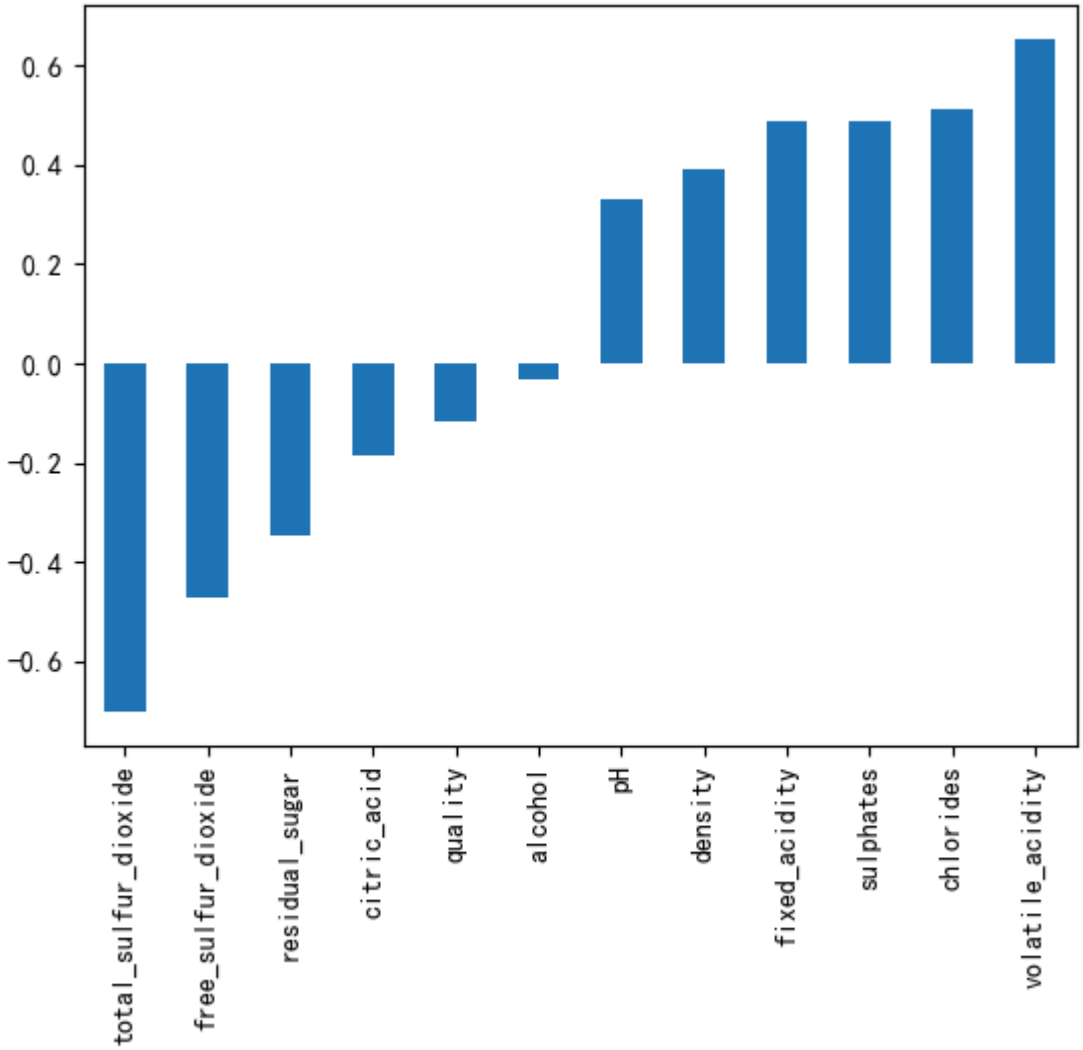
出[21]:

```
total_sulfur_dioxide   -0.700357
free_sulfur_dioxide    -0.471644
residual_sugar         -0.348821
citric_acid            -0.187397
quality                -0.119323
alcohol                -0.032970
pH                      0.329129
density                 0.390645
fixed_acidity           0.486740
sulphates               0.487218
chlorides               0.512678
volatile_acidity        0.653036
dtype: float64
```

在 [22]…

```
correlations.plot(kind='bar')
```

出[22]: <Axes: >



在 [23]…

```
# create X

fields = correlations.map(abs).sort_values().iloc[-2:].index
print(fields)

X = data[fields]
```

```
Index(['volatile_acidity', 'total_sulfur_dioxide'], dtype='object')
```

在 [24]…

```
# use MinMaxScaler to scale X
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)  # 缩放数据（输出为numpy数组）
X = pd.DataFrame(X_scaled, columns=fields)  # 转换回DataFrame并保留列名
```

第二步：线性SVC

训练LinearSVC分类器，绘制并观察其决策边界（可以尝试多种不同的参数设置）。绘制决策边界的函数参考了sklearn文档中的案例。

在 [25]…

```
# Helper function to plot a decision boundary.

def plot_decision_boundary(clf, X, y_color, h=0.01):
    # Set min and max values and generate a grid of points with gap h between th
    x_min, x_max = X[:, 0].min()-.1, X[:, 0].max()+.1
    y_min, y_max = X[:, 1].min()-.1, X[:, 1].max()+.1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

    # Predict the value for the whole gid
    Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # Plot the contour and the training examples
    fig, ax = plt.subplots(figsize=(10, 10))
    ax.contourf(xx, yy, Z, cmap=plt.cm.autumn_r, alpha=0.3)
    ax.scatter(X[:, 0], X[:, 1], c=y_color, cmap=plt.cm.autumn_r, alpha=1)
    ax.set(xlabel=fields[0], ylabel=fields[1], title=str(clf))
```

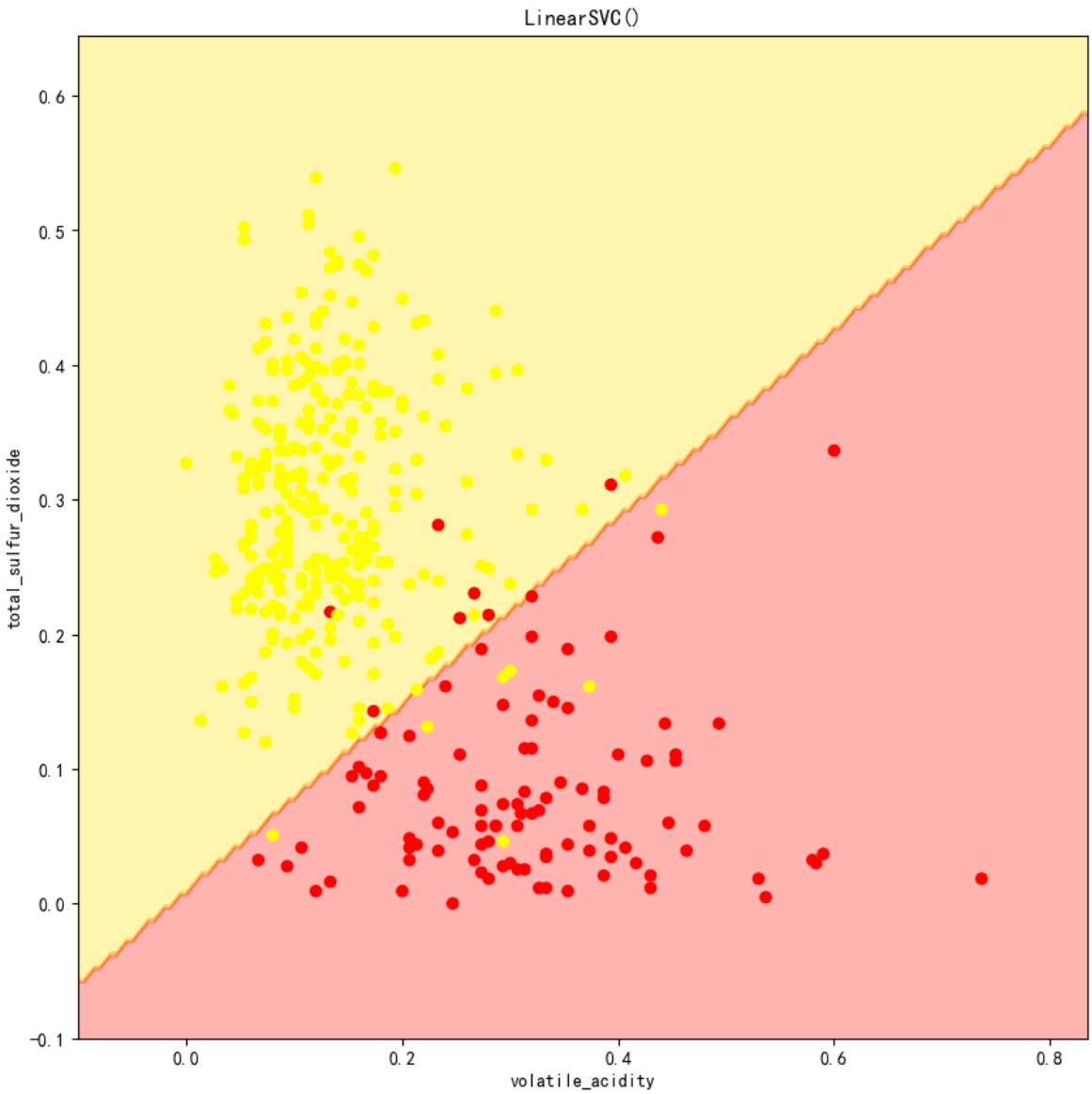
在 [26]…

```
# Pick 400 samples from X and get the corresponding y value. This is because ori
X_sample = X.sample(400, random_state=45)
y_sample = y.loc[X_sample.index]
# create y_color so that it has the value "red" for 1 and 'yellow' otherwise.
y_color = y_sample.map(lambda r: 'red' if r == 1 else 'yellow')
```

在 [27]…

```
# plot the decision boundary for LinearSVC
from sklearn.svm import LinearSVC
import numpy as np
import matplotlib.pyplot as plt

lsvc = LinearSVC()
lsvc.fit(X, y)

plot_decision_boundary(lsvc, X_sample.values, y_color.values, h=0.005)
```

```
D:\anaconda3\Lib\site-packages\sklearn\base.py:439: UserWarning: X does not have
valid feature names, but LinearSVC was fitted with feature names
  warnings.warn(
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_12496\595288183.py:16: UserWarning:
No data for colormapping provided via 'c'. Parameters 'cmap' will be ignored
  ax.scatter(X[:, 0], X[:, 1], c=y_color, cmap=plt.cm.autumn_r, alpha=1)
```



第三步：高斯核函数SVC

训练使用高斯核函数的SVC分类器，绘制并观察其决策边界随不同参数值的变化。

- 用以下不同的值分别创建高斯核函数SVC分类器，绘制并观察它们的决策边界：`gamma`

`gammas = [.5, 1, 2, 10]`

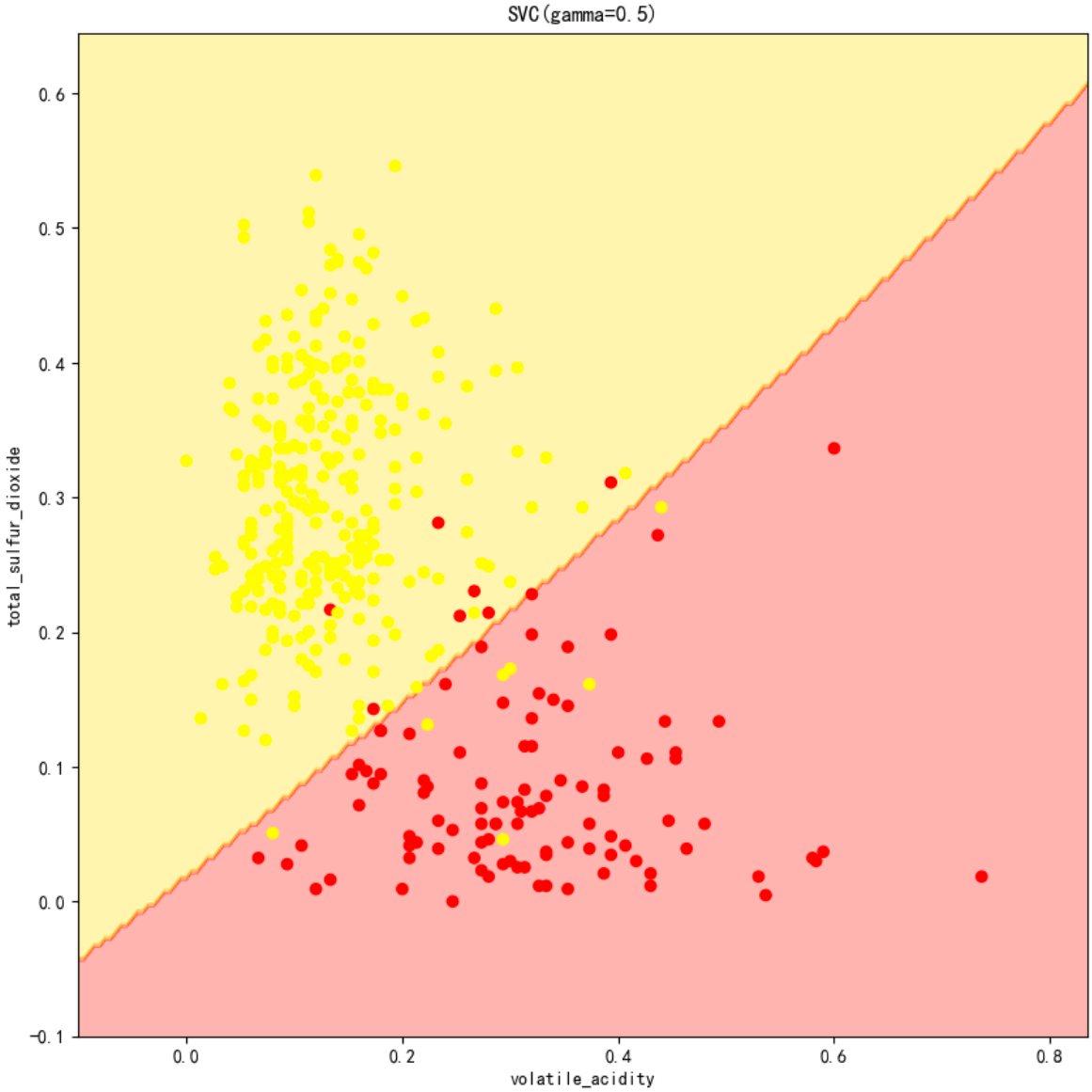
- 保持值不变（等于2），用以下不同的值创建高斯核函数SVC分类器，绘制并观察它们的决策边界：`gamma C`

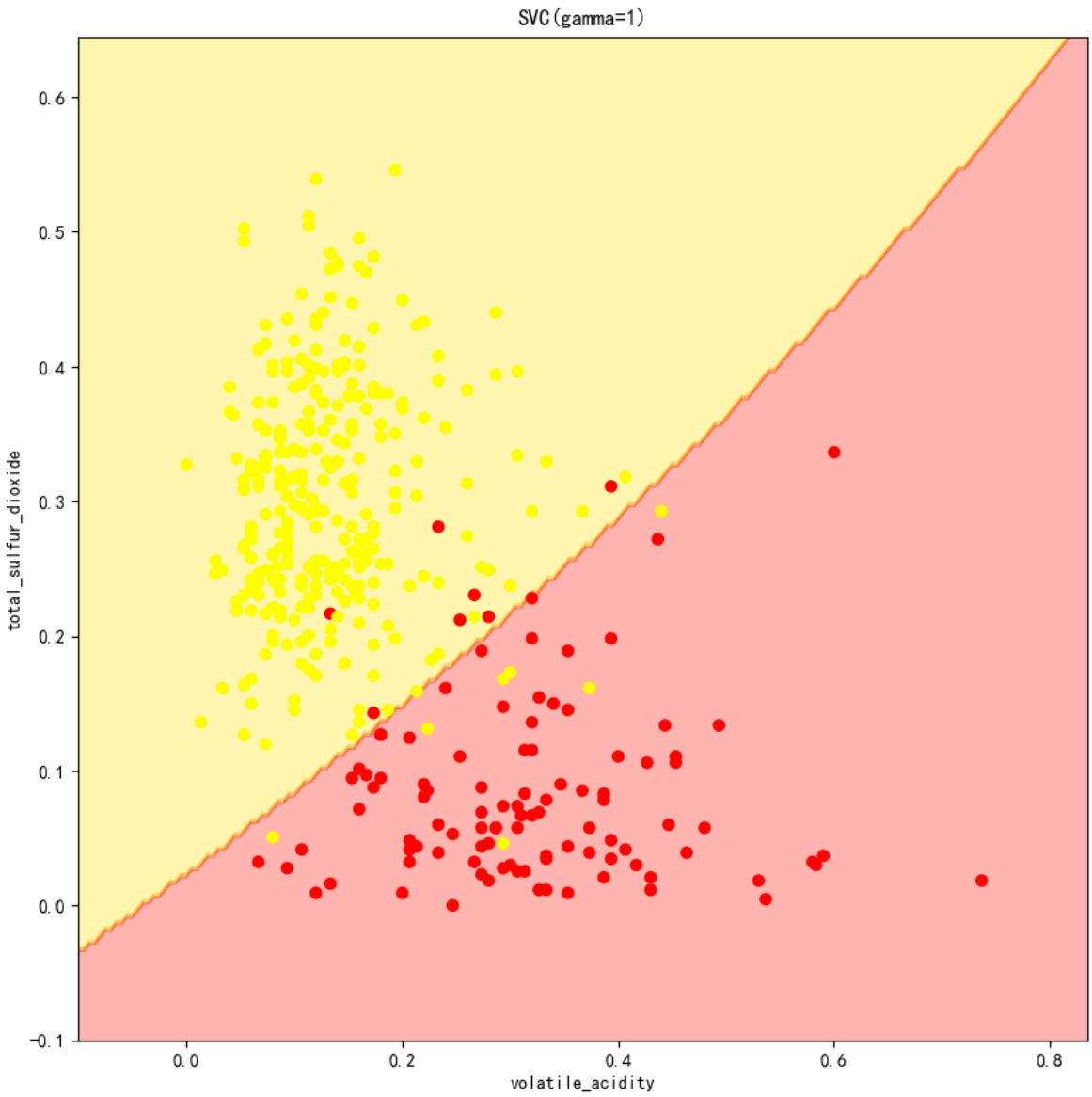
`Cs = [.1, 1, 10]`

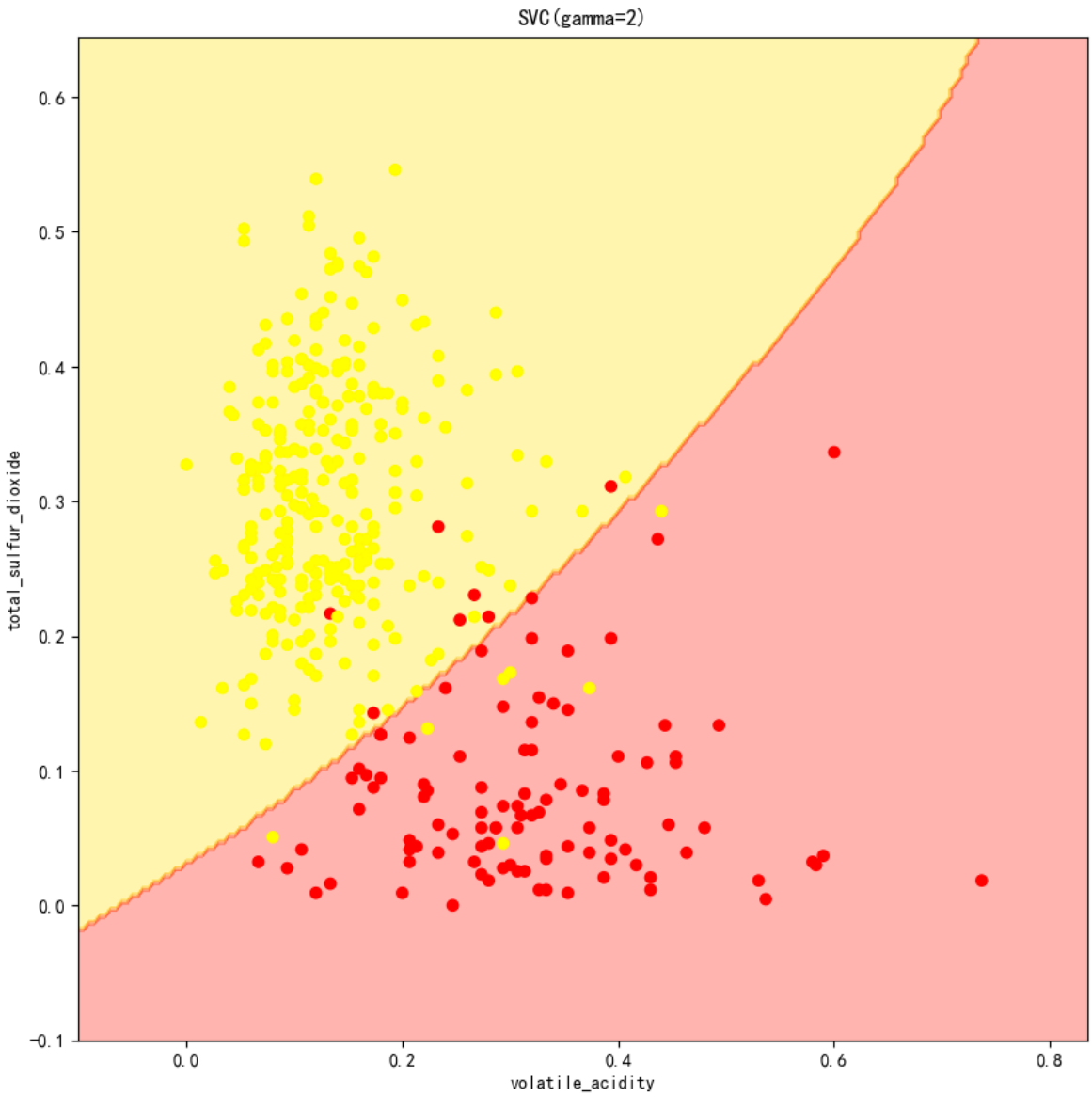
在 [28]…

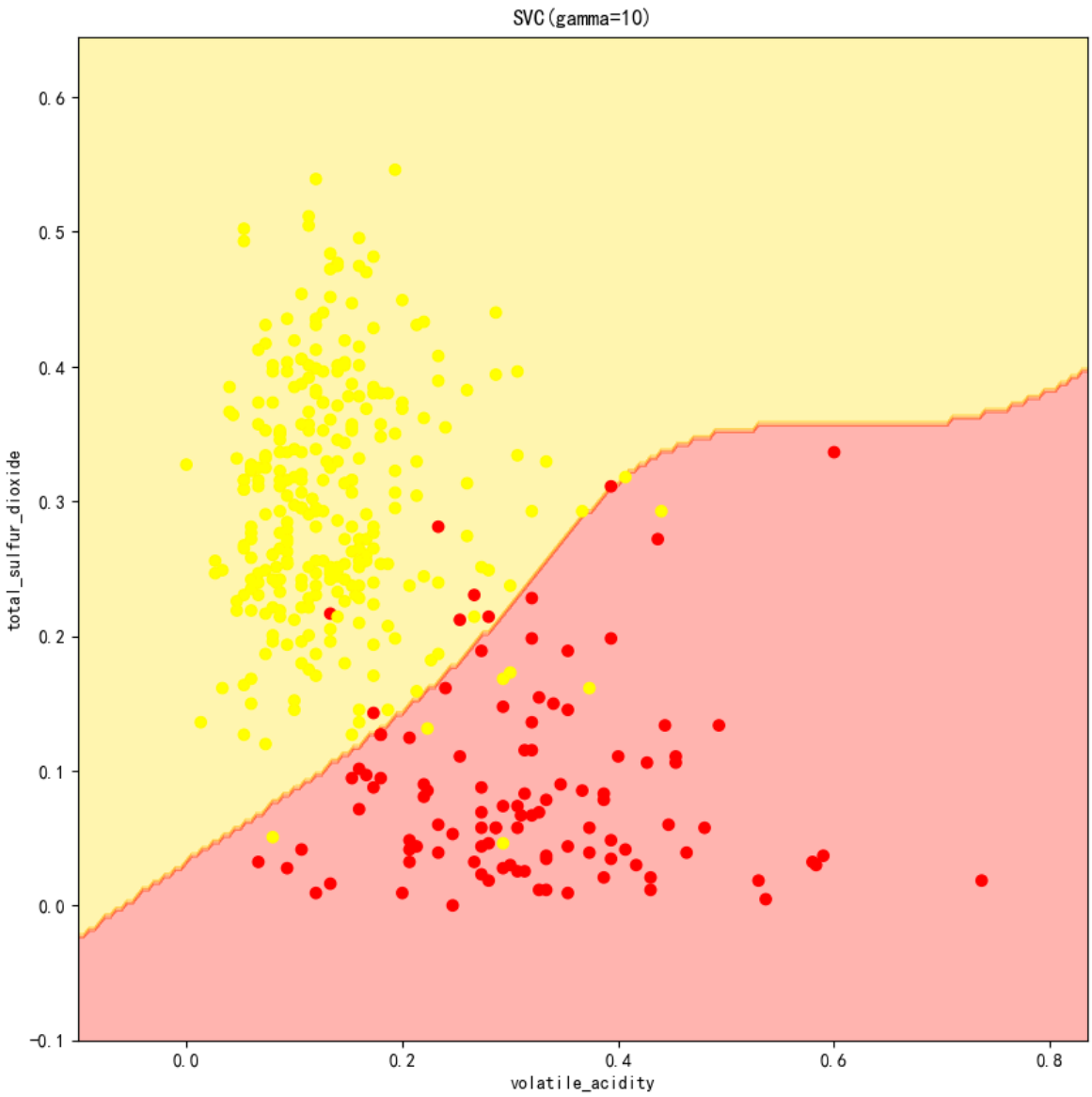
```
from sklearn.svm import SVC

gammas = [.5, 1, 2, 10]
for gamma in gammas:
    svc_Gaussian = SVC(kernel='rbf', gamma=gamma)
    svc_Gaussian.fit(X, y)
    plot_decision_boundary(svc_Gaussian, X_sample.values, y_color.values, h=0.00
```

```
D:\anaconda3\Lib\site-packages\sklearn\base.py:439: UserWarning: X does not have
valid feature names, but SVC was fitted with feature names
  warnings.warn(
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_12496\595288183.py:16: UserWarning:
No data for colormapping provided via 'c'. Parameters 'cmap' will be ignored
  ax.scatter(X[:, 0], X[:, 1], c=y_color, cmap=plt.cm.autumn_r, alpha=1)
D:\anaconda3\Lib\site-packages\sklearn\base.py:439: UserWarning: X does not have
valid feature names, but SVC was fitted with feature names
  warnings.warn(
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_12496\595288183.py:16: UserWarning:
No data for colormapping provided via 'c'. Parameters 'cmap' will be ignored
  ax.scatter(X[:, 0], X[:, 1], c=y_color, cmap=plt.cm.autumn_r, alpha=1)
D:\anaconda3\Lib\site-packages\sklearn\base.py:439: UserWarning: X does not have
valid feature names, but SVC was fitted with feature names
  warnings.warn(
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_12496\595288183.py:16: UserWarning:
No data for colormapping provided via 'c'. Parameters 'cmap' will be ignored
  ax.scatter(X[:, 0], X[:, 1], c=y_color, cmap=plt.cm.autumn_r, alpha=1)
D:\anaconda3\Lib\site-packages\sklearn\base.py:439: UserWarning: X does not have
valid feature names, but SVC was fitted with feature names
  warnings.warn(
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_12496\595288183.py:16: UserWarning:
No data for colormapping provided via 'c'. Parameters 'cmap' will be ignored
  ax.scatter(X[:, 0], X[:, 1], c=y_color, cmap=plt.cm.autumn_r, alpha=1)
```




SVC(gamma=1)
total_sulfur_dioxide
volatile_acidity
0.6
0.5
0.4
0.3
0.2
0.1
0.0
−0.1
0.0
0.2
0.4
0.6
0.8

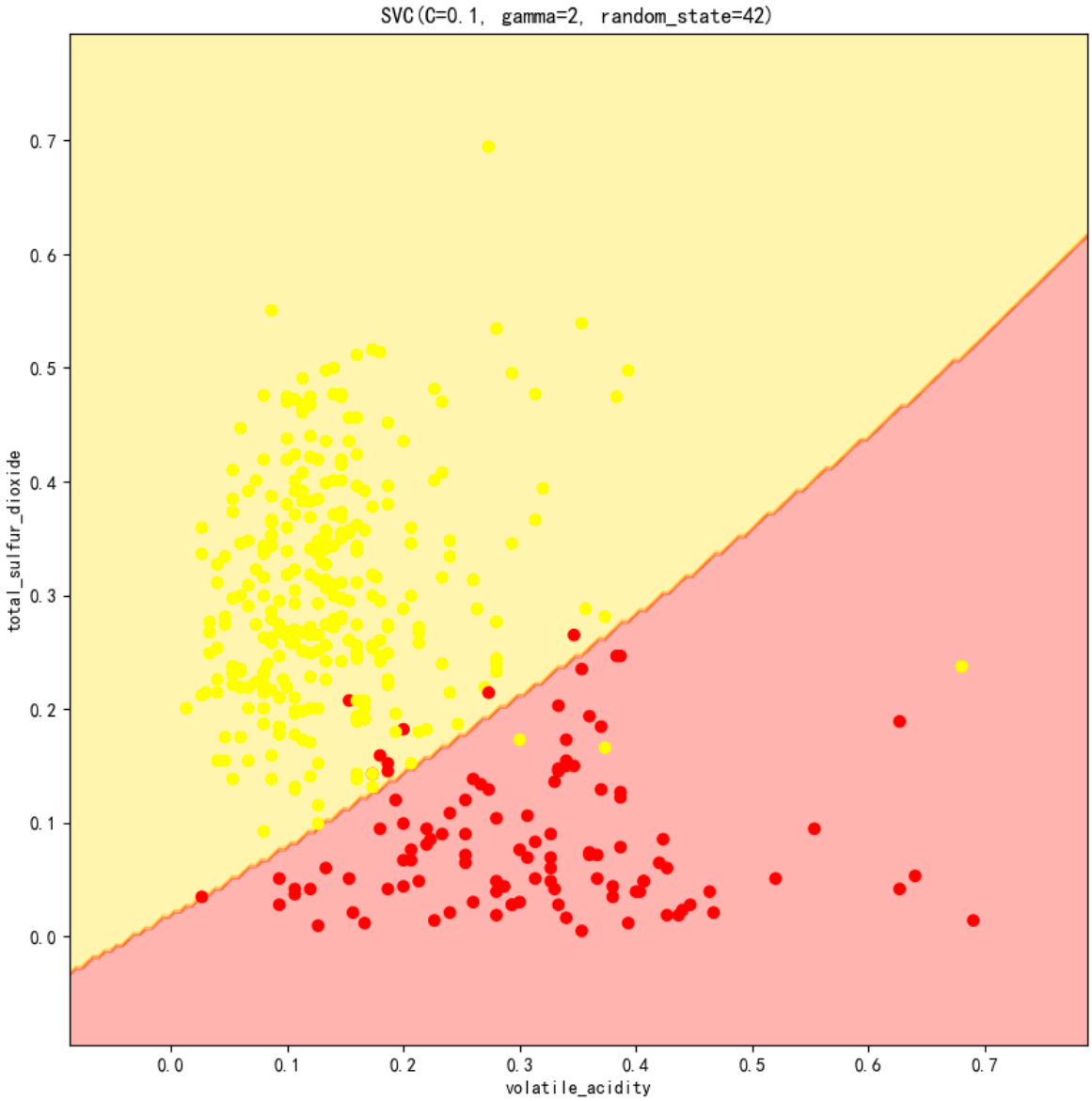

SVC(gamma=2)
total_sulfur_dioxide
volatile_acidity
0.6
0.5
0.4
0.3
0.2
0.1
0.0
−0.1
0.0
0.2
0.4
0.6
0.8

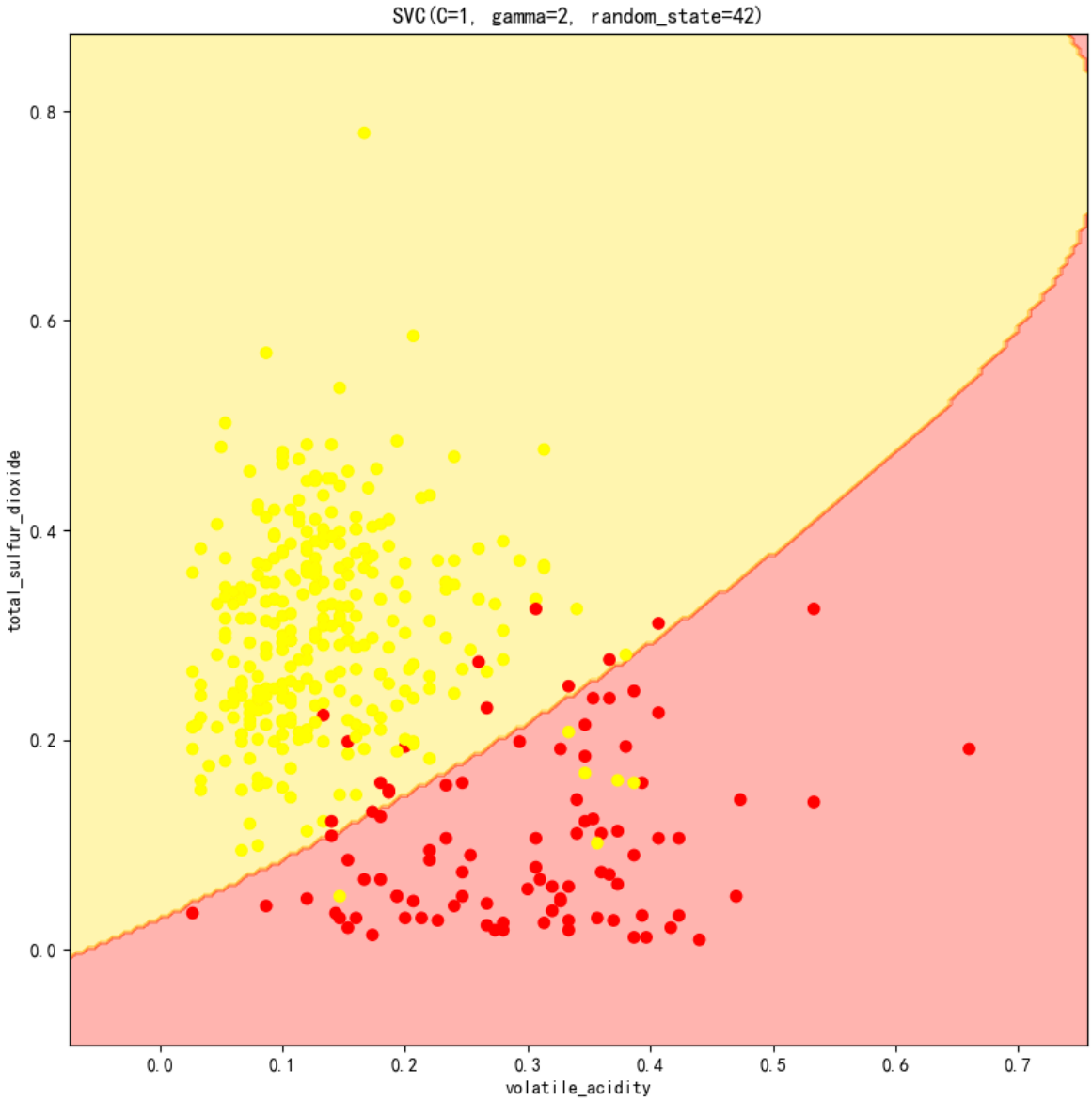


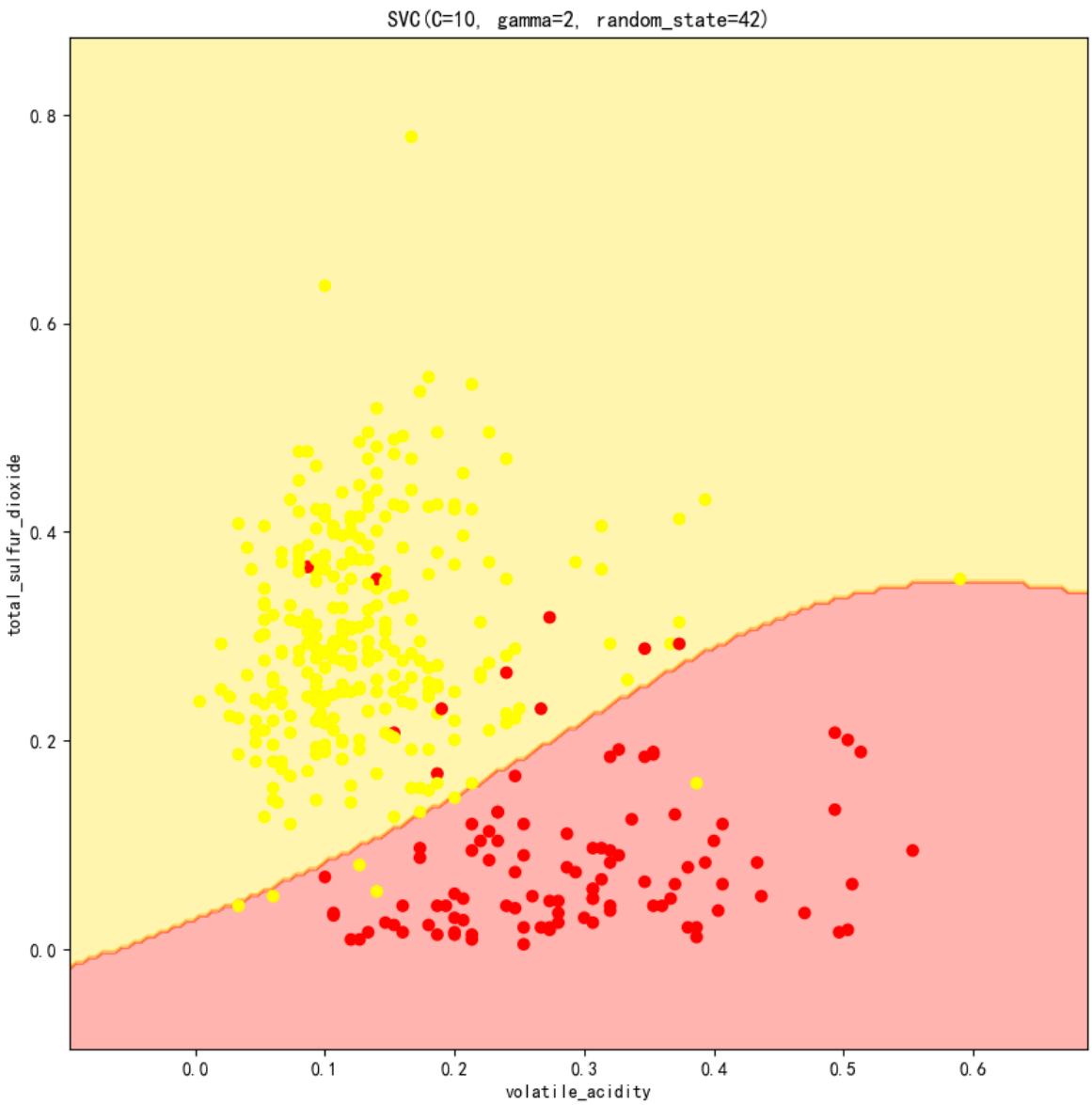
在 [29]…

```
Cs = [0.1, 1, 10]
# 对每个C值进行训练和绘图
for C in Cs:
    # 创建并训练SVC模型
    svc_gaussian = SVC(kernel='rbf', gamma=2, C=C, random_state=42)
    svc_gaussian.fit(X, y)

    # 从训练数据中随机抽样400个样本
    # 注意：使用X.index来确保索引一致性
    sample_indices = np.random.choice(X.index, 400, replace=False)
    X_sample = X.loc[sample_indices]
    y_sample = y.loc[sample_indices]

    # 创建颜色映射
    y_color = y_sample.map(lambda r: 'red' if r == 1 else 'yellow')

    # 绘制决策边界
    plot_decision_boundary(svc_gaussian, X_sample.values, y_color.values, h=0.00
```

```
D:\anaconda3\Lib\site-packages\sklearn\base.py:439: UserWarning: X does not have
valid feature names, but SVC was fitted with feature names
  warnings.warn(
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_12496\595288183.py:16: UserWarning:
No data for colormapping provided via 'c'. Parameters 'cmap' will be ignored
  ax.scatter(X[:, 0], X[:, 1], c=y_color, cmap=plt.cm.autumn_r, alpha=1)
D:\anaconda3\Lib\site-packages\sklearn\base.py:439: UserWarning: X does not have
valid feature names, but SVC was fitted with feature names
  warnings.warn(
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_12496\595288183.py:16: UserWarning:
No data for colormapping provided via 'c'. Parameters 'cmap' will be ignored
  ax.scatter(X[:, 0], X[:, 1], c=y_color, cmap=plt.cm.autumn_r, alpha=1)
D:\anaconda3\Lib\site-packages\sklearn\base.py:439: UserWarning: X does not have
valid feature names, but SVC was fitted with feature names
  warnings.warn(
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_12496\595288183.py:16: UserWarning:
No data for colormapping provided via 'c'. Parameters 'cmap' will be ignored
  ax.scatter(X[:, 0], X[:, 1], c=y_color, cmap=plt.cm.autumn_r, alpha=1)
```




SVC(C=1, gamma=2, random_state=42)
total_sulfur_dioxide
volatile_acidity
0.0
0.2
0.4
0.6
0.8
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7



第三步：多项式核函数SVC

训练使用3次方的多项式核函数的SVC分类器，绘制并观察其决策边界随不同参数值的变化：

- 用以下不同的C值创建多项式核函数SVC分类器，绘制并观察它们的决策边界：

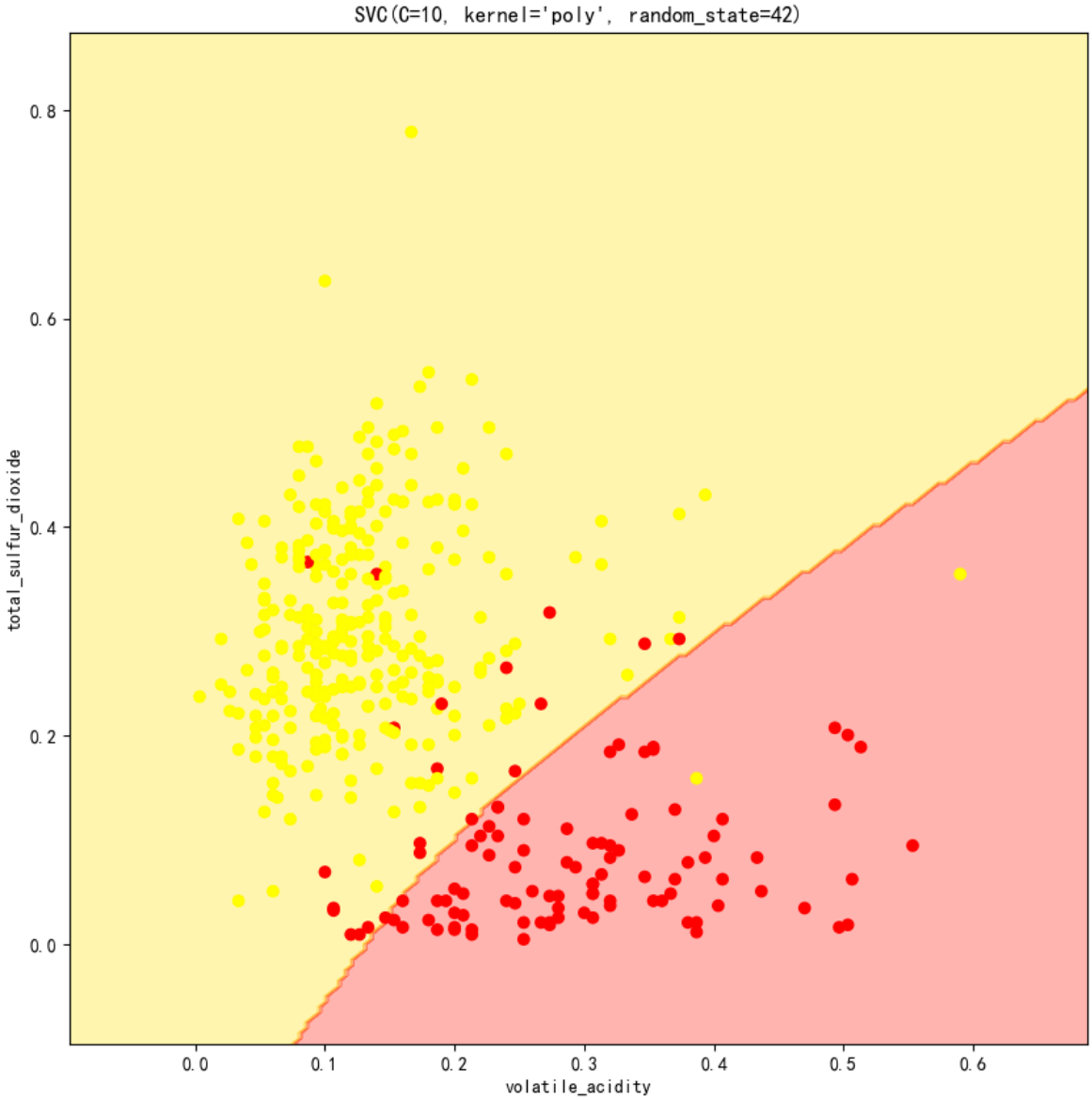
`Cs = [10,20,100,200]`

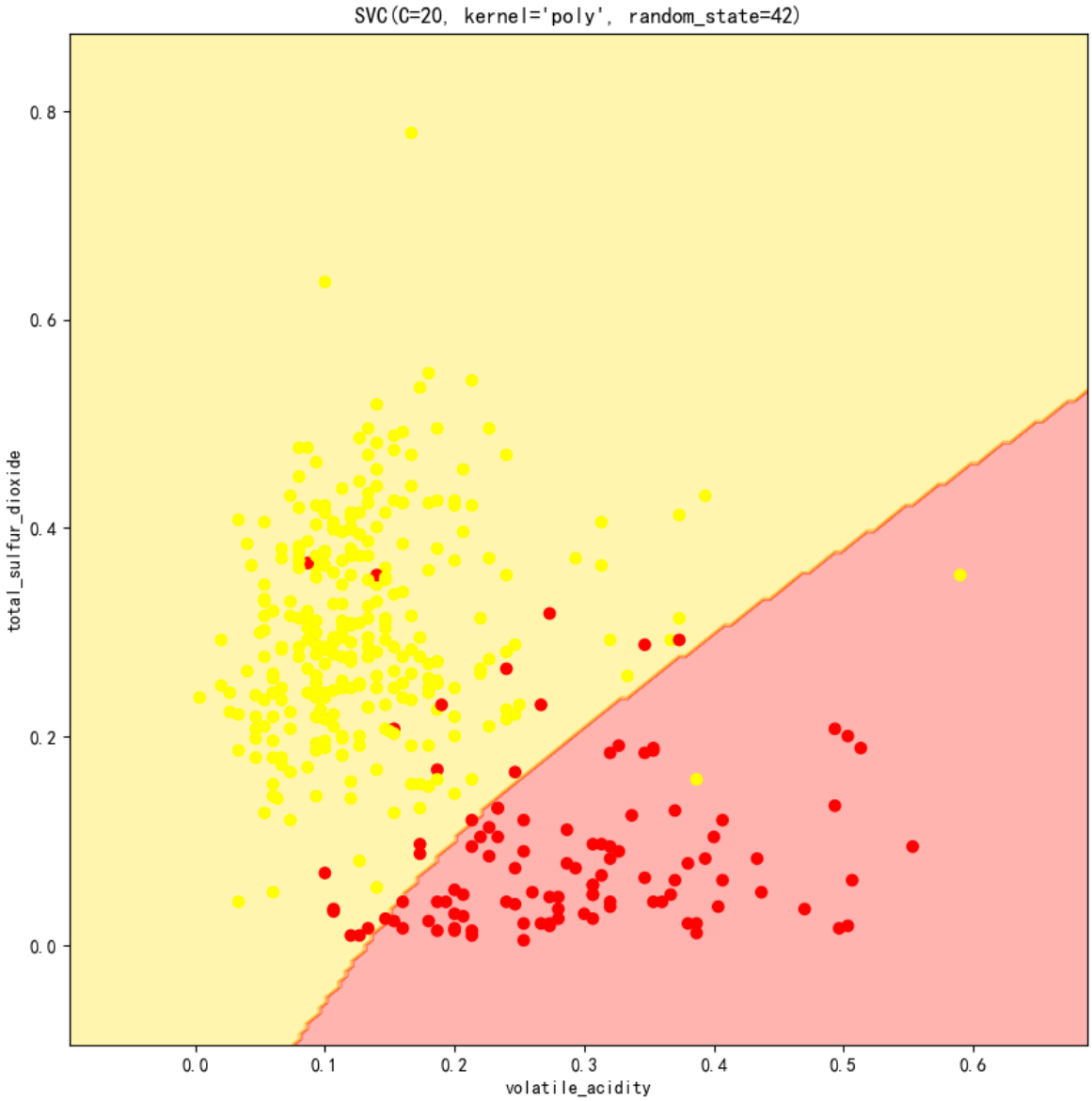
- 试着找出能产生最佳决策边界的C值

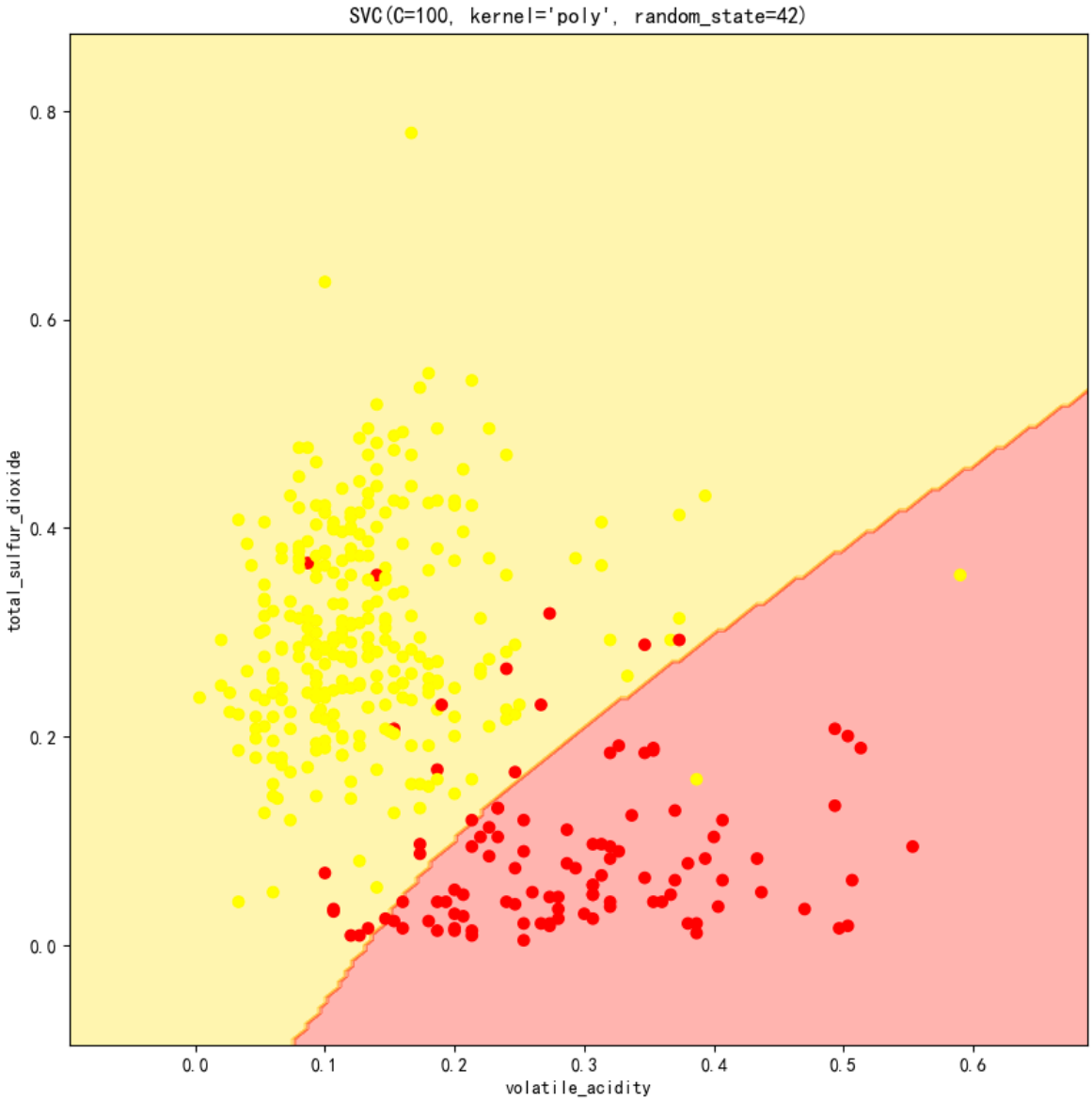
在 [30]…

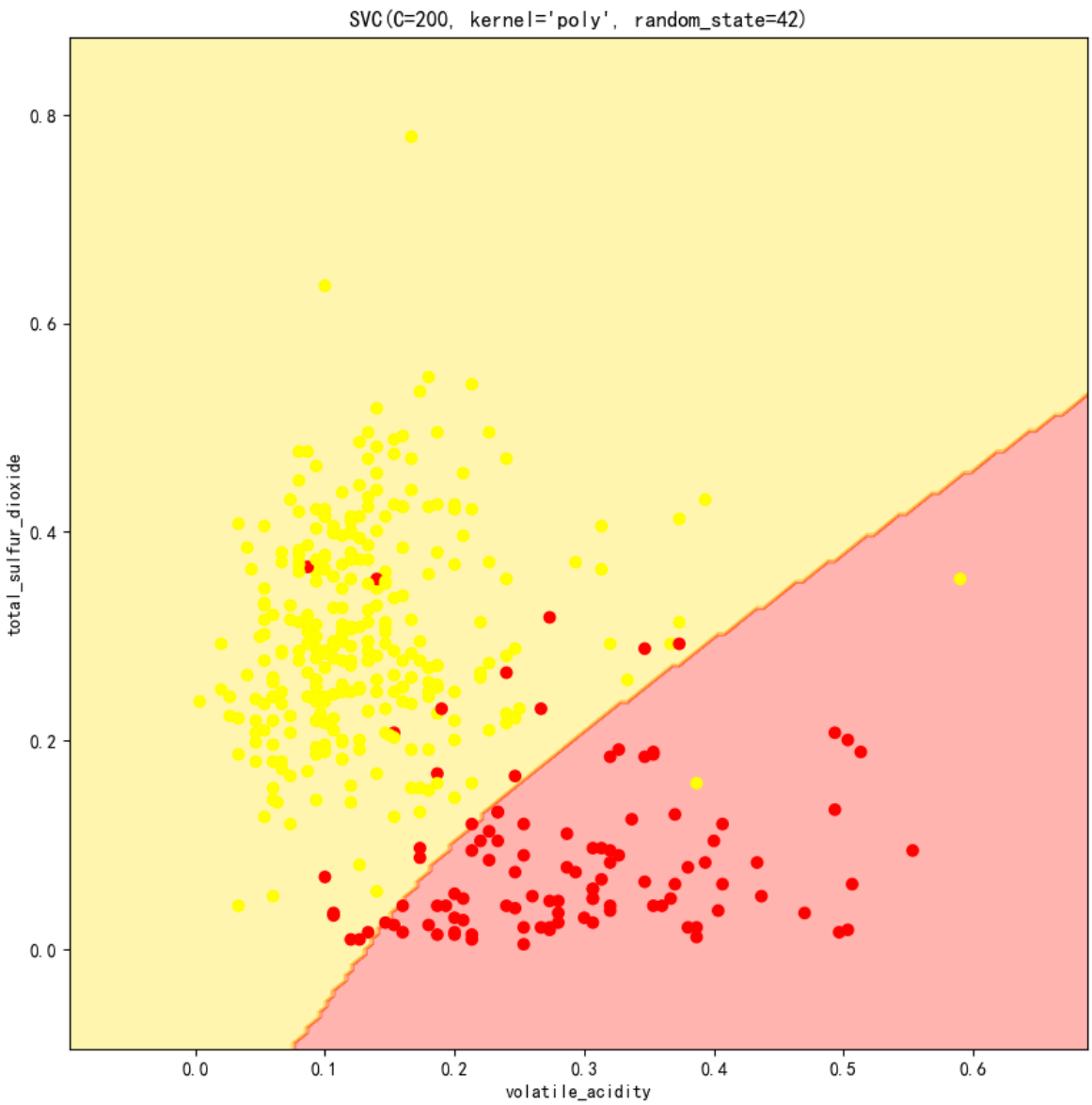
```
# Try with Polynomial kernel SVC
# 多项式核（3次），不同C值
Cs = [10, 20, 100, 200]
for C in Cs:
    svc_poly = SVC(kernel='poly', degree=3, C=C, random_state=42)  # 多项式核，d
    svc_poly.fit(X, y)
    plot_decision_boundary(svc_poly, X_sample.values, y_color.values, h=0.005)
```

```
D:\anaconda3\Lib\site-packages\sklearn\base.py:439: UserWarning: X does not have
valid feature names, but SVC was fitted with feature names
  warnings.warn(
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_12496\595288183.py:16: UserWarning:
No data for colormapping provided via 'c'. Parameters 'cmap' will be ignored
  ax.scatter(X[:, 0], X[:, 1], c=y_color, cmap=plt.cm.autumn_r, alpha=1)
D:\anaconda3\Lib\site-packages\sklearn\base.py:439: UserWarning: X does not have
valid feature names, but SVC was fitted with feature names
  warnings.warn(
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_12496\595288183.py:16: UserWarning:
No data for colormapping provided via 'c'. Parameters 'cmap' will be ignored
  ax.scatter(X[:, 0], X[:, 1], c=y_color, cmap=plt.cm.autumn_r, alpha=1)
D:\anaconda3\Lib\site-packages\sklearn\base.py:439: UserWarning: X does not have
valid feature names, but SVC was fitted with feature names
  warnings.warn(
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_12496\595288183.py:16: UserWarning:
No data for colormapping provided via 'c'. Parameters 'cmap' will be ignored
  ax.scatter(X[:, 0], X[:, 1], c=y_color, cmap=plt.cm.autumn_r, alpha=1)
D:\anaconda3\Lib\site-packages\sklearn\base.py:439: UserWarning: X does not have
valid feature names, but SVC was fitted with feature names
  warnings.warn(
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_12496\595288183.py:16: UserWarning:
No data for colormapping provided via 'c'. Parameters 'cmap' will be ignored
  ax.scatter(X[:, 0], X[:, 1], c=y_color, cmap=plt.cm.autumn_r, alpha=1)
```




SVC(C=20, kernel='poly', random_state=42)
total_sulfur_dioxide
volatile_acidity
0.0
0.2
0.4
0.6
0.8
0.0
0.1
0.2
0.3
0.4
0.5
0.6


SVC(C=100, kernel='poly', random_state=42)
total_sulfur_dioxide
volatile_acidity
0.0
0.2
0.4
0.6
0.8
0.0
0.1
0.2
0.3
0.4
0.5
0.6



第四步：Nystroem+SGD

比较径向基核函数的SVC与Nystroem+SGD方法的训练速度。使用Jupyter Notebook提供的魔术命令 `%timeit` 输出一个代码单元的运行时间。

- 恢复最初的葡萄酒质量数据
- 从“color”列生成，其他列生成 `y X`
- 用输出一个径向基核函数SVC分类器的训练时间 `%%timeit`
- 用输出下面作的运行时间：用Nystroem fit_transform数据，然后训练一个SGDClassifier `%%timeit`

Nystroem+SGD一般会更快一些，尤其是数据集越大时优势越明显。

- 把X复制5份并串接起来，生成一个五倍大的数据集
- 相应地，把y也拷贝5份并串接起来，变成原来的五倍
- 再次比较上面两种方法的训练速度

在 [32]…

```
from sklearn.kernel_approximation import Nystroem
from sklearn.svm import SVC
```

```
from sklearn.linear_model import SGDClassifier

y = data.color == 'red'
X = data[data.columns[:-1]]

svc = SVC(kernel='rbf')
nystroem = Nystroem(kernel='rbf')
sgd = SGDClassifier()
```

在 [33]…

```
%%timeit
svc.fit(X, y)
```

```
291 ms ± 7.51 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

在 [34]…

```
%%timeit
X_transformed = nystroem.fit_transform(X)
sgd.fit(X_transformed, y)
```

```
61.3 ms ± 1.97 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

在 [35]…

```
X2 = pd.concat([X]*5)
y2 = pd.concat([y]*5)

print(X2.shape)
print(y2.shape)
```

```
(32485, 12)
(32485,)
```

在 [36]…

```
%timeit svc.fit(X2, y2)
```

```
7.01 s ± 426 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

在 [37]…

```
%%timeit
X2_transformed = nystroem.fit_transform(X2)
sgd.fit(X2_transformed, y2)
```

```
210 ms ± 8.75 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

第五步：自动调参

使用GridSearchCV自动调优SVC中的超参数：

- 使用全部数据集
- y 被定义为color = 'red'
- 其他列定义为 X
- 划分训练集和测试集
- 设置交叉验证的参数值，以及许多不同gamma 和 C 的值
- 用GridSearchCV自动运行所有可能的参数值组合
- 显示各种参数值组合运行得分的平均值和标准差
- 输出最佳参数值组合

在 [38]…

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
param_grid = {
    'C': [0.1, 1, 10],
```

```
    'gamma': [0.1, 1, 10],
    'kernel': ['rbf', 'linear']
}

# 使用随机搜索替代网格搜索
random_search = RandomizedSearchCV(
    SVC(random_state=42),
    param_distributions=param_grid,
    n_iter=10,  # 仅尝试10种组合
    cv=3,       # 3折交叉验证
    n_jobs=2,   # 使用2个CPU核心
    verbose=2,  # 显示详细进度
    scoring='accuracy'
)

# 在降采样的训练集上执行搜索
X_train_sample = X_train.sample(frac=0.5, random_state=42)
y_train_sample = y_train.loc[X_train_sample.index]

# 执行搜索并记录时间
%time random_search.fit(X_train_sample, y_train_sample)
```

```
Fitting 3 folds for each of 10 candidates, totalling 30 fits
CPU times: total: 516 ms
Wall time: 10.6 s
```

出[38]:

▸ 随机搜索CV

▸ 估算器：SVC

▸ SVC

第六步：预测与评价

用上一步得到的最佳的模型

- 预测测试集中的数据
- 显示混淆矩阵、精度和分类报告

在 [41]…

```
from sklearn.metrics import confusion_matrix, accuracy_score, classification_rep
best_model = random_search.best_estimator_

# 在完整测试集上评估
%time y_pred = best_model.predict(X_test)

print("混淆矩阵：\n", confusion_matrix(y_test, y_pred))
print("准确率：", accuracy_score(y_test, y_pred))
print("分类报告：\n", classification_report(y_test, y_pred))
```

```
CPU times: total: 15.6 ms
Wall time: 5.67 ms
混淆矩阵：
 [[951   8]
 [ 17 324]]
准确率： 0.9807692307692307
分类报告：
               precision    recall  f1-score   support

       False       0.98      0.99      0.99       959
        True       0.98      0.95      0.96       341

    accuracy                           0.98      1300
   macro avg       0.98      0.97      0.97      1300
weighted avg       0.98      0.98      0.98      1300
```

在 [ ] …